

Ejercicio: Demostración Master Method

- José Antonio Mérida Castejón
- Jonathan Alejandro Díaz Tahuite
- Luis Francisco Padilla Juarez

15 de febrero de 2026

Parte 1: Demostración de la cota inferior

Queremos demostrar que:

$$\sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) = \Omega(f(n))$$

1. Se extrae el primer término ($j = 0$) de la sumatoria:

$$\sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) = a^0 f\left(\frac{n}{1}\right) + \sum_{j=1}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

2. Se simplifica el término $j = 0$ sabiendo que $a^0 = 1$:

$$= f(n) + \sum_{j=1}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

3. Dado que asumimos costos no negativos, la sumatoria restante es mayor o igual a 0:

$$\sum_{j=1}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) \geq 0$$

4. Por lo tanto, la suma total es mayor o igual que el primer término por sí solo:

$$\text{Total} \geq f(n) + 0$$

5. Por definición de cota inferior asintótica:

$$\sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) = \Omega(f(n))$$

Parte 2: Simplificación de la Complejidad Total

Pregunta: ¿Por qué $\Theta(n^{\log_b a}) + \Theta(f(n)) = \Theta(f(n))$?

Explicación:

En el análisis de algoritmos, cuando sumamos dos complejidades, el resultado es del orden del término mayor:

$$\Theta(A) + \Theta(B) = \Theta(\max(A, B))$$

Para que este caso aplique, debe cumplirse la condición de que $f(n)$ crezca polinómicamente más rápido que la parte recursiva $n^{\log_b a}$.

La suposición formal es:

$$f(n) = \Omega(n^{\log_b a + \epsilon}) \quad \text{para algún } \epsilon > 0$$

Dado que $f(n)$ es el término dominante, el término $n^{\log_b a}$ se vuelve insignificante en la suma para n grande.

Resultado:

$$T(n) = \Theta(f(n))$$